Alongside every month

Senior Data Scientist roadmap

B17 Testing and performance

B17-L1 Unit tests for data code that catch real bugs

Test edges on purpose, with tiny explicit inputs, one behaviour per test, and prove each test can fail.

```python
import pytest


def test_share_zero_total_is_none():
    assert share(3, 0) is None


@pytest.mark.parametrize(("part", "total", "expected"), [(1, 4, 0.25), (0, 5, 0.0), (5, 5, 1.0)])
def test_share_values(part, total, expected):
    assert share(part, total) == expected
```

A test worth writing	One that fails when the code is wrong. Break the code once to prove it.
Edge cases for data code	Empty input, missing values, zero denominators, one row, duplicate keys.
Compare DataFrames in tests	pd.testing.assert_frame_equal: values, dtypes, columns and index.
Compare floats in tests	pytest.approx(expected).

B17-L2 Property-based tests with Hypothesis

State a rule that must always hold, generate inputs from the domain the function promises, and let Hypothesis find and shrink the counterexample.

```python
from hypothesis import given, strategies as st

@given(st.lists(st.tuples(st.floats(-1e6, 1e6), st.floats(0, 10)), min_size=1))
def test_weighted_mean_within_range(pairs):
    values = [v for v, _ in pairs]
    weights = [w for _, w in pairs]
    m = weighted_mean(values, weights)
    assert m is None or min(values) - 1e-6 <= m <= max(values) + 1e-6
```

Property-based test	A rule checked against many generated inputs, with failing inputs shrunk to a minimal case.
Four data-code properties	Bounds, invariance, round trip, idempotence, grain, and agreement with a slow version.
st.floats() default	Includes nan and infinities. Bound it or pass allow_nan=False on purpose.
Hypothesis example database	.hypothesis/ replays past failures. Keep it out of Git.

B17-L3 Measure before you optimise

Profile to find the cost, time the change with perf_counter or timeit, keep setup out of the timing, and fix the biggest cost first.

```
import cProfile, pstats
cProfile.run("main()", "profile.out")
pstats.Stats("profile.out").sort_stats("cumulative").print_stats(10)
```

Find where time goes	cProfile with pstats sorted by cumulative, or %prun in a notebook.
Time a small change	timeit.repeat or perf_counter. Report the minimum per call.
Measure memory	tracemalloc peak, and df.memory_usage(deep=True) for pandas.
The order of work	Profile, fix the biggest cost, re-measure, keep a test around the change.

B19 Testing in depth

B19-L1 How pytest finds, runs and reports your tests

Name files and functions so pytest collects them, assert instead of printing, and pin expected errors with pytest.raises and match.

```
E       assert 0.3333333333333333 == 0.33
E        +  where 0.3333333333333333 = share(1, 3)
```

```
with pytest.raises(ValueError, match="negative weight"):
    normalise([2.0, -1.0])
```

What pytest collects by default	Files `test_*.py` or `*_test.py`, functions starting with `test`, and test methods in `Test` classes without `__init__`.
Test that code raises the right error	with pytest.raises(ValueError, match="pattern"): call(). match searches the message with a regular expression.
Run one test, or only failures	pytest path::test_name for one test. -k expr selects by name. -x stops at the first failure. --lf reruns last failures.
pytest exit codes	0 all passed, 1 a test failed, 5 no tests collected.

B19-L2 Fixtures, shared setup without shared state

Fixtures give each test fresh data by default. Share a fixture only when it is expensive and nothing changes it.

```
@pytest.fixture
def respondents() -> pd.DataFrame:
    return pd.DataFrame({"respondent_id": ["R1", "R2"], "final_weight": [1.0, 3.0]})


def test_total_weight(respondents: pd.DataFrame) -> None:
    assert respondents["final_weight"].sum() == 4.0
```

```
@pytest.fixture
def make_respondents():
    def make(weights: list[float]) -> pd.DataFrame:
        ids = [f"R{i}" for i in range(1, len(weights) + 1)]
        return pd.DataFrame({"respondent_id": ids, "final_weight": weights})
    return make
```

What a fixture is	A function decorated with @pytest.fixture. A test gets its value by naming it as a parameter.

Fixture setup and teardown	Code before yield is setup. Code after yield is teardown, and it runs even when the test fails.
When to widen a fixture's scope	Only for expensive objects that no test changes. A shared object that a test changes makes results depend on test order.
Factory fixture	A fixture that returns a function building fresh data with defaults, so each test states only what matters.
Share fixtures across files	Put them in conftest.py. Tests in that folder and below use them without an import.

B19-L3 Parametrize, one test and many readable cases

One row per case, with both sides of every threshold and the missing value, each named so a failure explains itself.

```python
@pytest.mark.parametrize(
    ("age", "expected"),
    [
        pytest.param(34, "16-34", id="below-35"),
        pytest.param(35, "35-54", id="at-35"),
        pytest.param(None, None, id="missing"),
    ],
)
def test_age_band(age: int | None, expected: str | None) -> None:
    assert age_band(age) == expected
```

```python
@pytest.fixture(params=["pandas", "polars"])
def backend(request: pytest.FixtureRequest) -> str:
    return request.param
```

Run one test over a table of cases	@pytest.mark.parametrize(("age", "expected"), [...]). Each row is its own test.
Name a parametrized case	pytest.param(34, "16-34", id="below-35") reports as test_age_band[below-35].
Mark a known bug on one row	pytest.param(..., marks=pytest.mark.xfail(reason=..., strict=True)). strict fails the run once the row passes.
Run every test once per implementation	@pytest.fixture(params=[...]) and return request.param.
Rows to pick for a threshold	Just below it, on it, and a missing value.

B19-L4 Testing pandas and NumPy code

Compare whole frames with assert_frame_equal against an expected frame you write out, and copy the input to prove the function leaves it alone.

What assert_frame_equal checks	Values (missing values in the same place count as equal), dtypes, column names and order, and the index. Floats with rtol 1e-5.
Compare rows whatever their order	Sort both frames by the key columns, reset both indexes, then assert_frame_equal(..., check_like=True).
Why a filtered result fails on the index	Filtering keeps the old labels, such as [0, 2]. Reset the index in the function if it promises a fresh one.
assert_allclose and an expected 0	Defaults are rtol=1e-7, atol=0. A relative tolerance of 0 accepts nothing, so add atol.

Test that a function leaves its input alone	before = df.copy(), call the function, then assert_frame_equal(df, before).

B19-L5 Testing statistical and random code

Check exact values against an oracle, seed every random draw, and size simulation tolerances from the standard error.

```python
from statsmodels.stats.proportion import proportion_confint

expected = proportion_confint(12, 40, alpha=0.05, method="wilson")
np.testing.assert_allclose(wilson_interval(12, 40), expected, rtol=1e-6)
```

Oracle test for a statistic	On a small input, compare with an independent library, such as statsmodels proportion_confint, using assert_allclose.
Make random code testable	Pass a np.random.Generator in. Tests pass np.random.default_rng(seed), often from a fixture.
Tolerance for a simulated coverage	Standard error sqrt(p(1 - p) / sims). Allow about four standard errors: 0.931 to 0.969 for 0.95 over 2,000 simulations.
An assertion never to write	p_value < 0.05 on random data. It fails by chance about one run in twenty.

B19-L6 Testing machine learning pipelines

Test contracts, leakage, determinism and a margin over a baseline on fixed data, not one exact score.

Leakage test for a pipeline	Fit on the training split, then assert pipe.named_steps["standardscaler"].mean_ equals X_train.mean(axis=0).
Contract tests for a classifier	predict_proba shape is (n_rows, n_classes), each row sums to 1, values lie in [0, 1].
Performance test that is not fragile	On fixed data from make_classification(random_state=0), assert the model beats DummyClassifier(strategy="prior") by a margin.
Behavioural test: row order	Shuffle with a seeded permutation and check each row's prediction is unchanged: compare with original[perm].

B19-L7 Isolate I/O with fakes, monkeypatch and mocks

Pass I/O in as a parameter where you can, use monkeypatch for settings, and patch a name where the code looks it up.

```python
class FakeResponse:
    status_code = 200

    def json(self) -> dict[str, int]:
        return {"respondents": 812}


assert wave_count("W14", get=lambda url, timeout: FakeResponse()) == 812
```

Easiest way to test code that does I/O	Pass the dependency in as a parameter, such as get or now. Tests pass a small fake.
Change an environment variable for one test	monkeypatch.setenv(name, value) or monkeypatch.delenv(name, raising=False). pytest restores it afterwards.

Mock that fails on the second call	Mock(side_effect=[result, ConnectionError("down")]). call_count counts both calls.
Where to patch	Where the code looks the name up. After from requests import get in reports.py, patch reports.get.

B19-L8 Coverage, markers and a test suite you can trust in CI

Measure branch coverage and set a floor, register markers strictly, fix flaky tests at the cause, and remember that coverage says what ran, not what was checked.

```
pytest --cov=survey_stats --cov-branch --cov-report=term-missing --cov-fail-under=90
```

```
[tool.pytest.ini_options]
addopts = "--strict-markers"
markers = ["slow: takes more than a few seconds"]
```

Line coverage against branch coverage	Line: did each line run? Branch (--cov-branch): did each if go both ways? A missed jump shows as 2->4 in Missing.
Fail CI when coverage drops	pytest --cov=pkg --cov-branch --cov-fail-under=90. Start near today's level and raise it.
Catch marker typos	Register markers under markers = [...] and add --strict-markers to addopts. pytest 9 also offers strict = true.
What coverage cannot tell you	Whether tests check anything. Mutation testing (mutmut 3.8) changes the code and reports changes no test catches.
A flaky test	Fix the cause: seed, shared state, clock or network. A retry hides it.

B18 Documentation and communication

B18-L1 Docstrings and READMEs that answer the next person's questions

Docstrings state grain, units, missing values and what None means. The README states purpose, quick start, inputs, config, tests and owners.

```
def weighted_share(df: pd.DataFrame, value: str, weight: str) -> float | None:
    """Weighted mean of a 0/1 column. Grain: one row per respondent.

    Args:
        df: Respondent-level data.
        value: 0/1 column to average.
        weight: Non-negative weight column. Rows with a missing value or weight are dropped.

    Returns:
        The weighted share, or None when no rows remain or weights sum to 0.

    Raises:
        KeyError: If a named column is missing.
    """
```

Google-style docstring sections	One-line summary, then Args:, Returns:, Raises:.
What a data docstring must state	Grain, units, missing and zero handling, and what None means.
README sections	Purpose, quick start, inputs and outputs, configuration, tests, owners.

Keeping docs true	Update docstrings in the same commit as the code. Test the quick start where possible.

B18-L2 Model cards that state limits as clearly as scores

A model card states purpose, limits, data, honest metrics, subgroup results and monitoring, and is reviewed with every model change.

Model card sections	Details, intended use, out-of-scope uses, training data, evaluation, subgroups, limitations, monitoring.
How to report a metric on a card	With test period, prevalence and row count.
Why out-of-scope uses	They stop reuse for decisions the model was never validated for.
Keep cards honest	Template with required sections, reviewed in the model's pull request, checked in CI.

B18-L3 Decision logs, so the reasons outlive the meeting

One short, dated entry per decision, with options, consequences and status, linked from the code it explains, and superseded rather than edited.

```
## D-007: Trim survey weights at 5
Date: 2026-09-12. Status: accepted.
Context: Four respondents had weights above 12 and moved brand share by 4 points.
Options considered:
- No trimming
- Trim at 5, redistribute to keep the weighted total
- Rake with bounded weights
Decision: Trim at 5 with redistribution. Raking deferred until the new panel design.
Consequences: Design effect falls from 2.1 to 1.6. Small bias towards the sample
  profile. Review at wave 20.
```

Parts of a decision entry	ID and title, date and status, context, options considered, decision, consequences.
When a decision changes	Add a new entry. Mark the old one superseded and link to the new one.
Where and when to write	In the repository, in the same pull request as the change.
Make it findable	Link from the config value, the model card and the assumptions log.